
# Property Valuation — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 21-March-2026

David K Kelly

## Legal Notice

## PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

# 1 Test Results — Property Valuation (MKM-PV-001)

Tests run on **21 March 2026 at 07:54:55**.

Table 1: Test Results Summary — Property Valuation (MKM-PV-001)

| | |
|---|---|
| Total Tests | 55 |
| Passed | 55 |
| Failed | 0 |
| Skipped | 0 |
| Duration | 11.85s |

Table 2: Detailed Test Results — Property Valuation

| Test Description | Acceptance Criteria | Result | Time |
|---|---|---|---|
| Line 541: non-dict schema → returns {}. | `result == {}` | Pass | 0.000s |
| Line 544-545: 'type'/'options'/'description' fields are skipped. | `"type" not in result; "options" not in result` | Pass | 0.000s |
| Calculate insurance premium flood risk high | `premium > 0` | Pass | 0.065s |
| Line 214: area=None → computes it first. | `isinstance(premium, float); premium > 0` | Pass | 0.066s |
| Line 211: property_value=None → computes it first. | `isinstance(premium, float); premium > 0` | Pass | 0.124s |
| Calculate monthly rent all types | `800 <= rent <= 15_000` | Pass | 0.000s |
| Lines 165-166: area=None → computes it first. | `isinstance(rent, float); rent > 0` | Pass | 0.000s |
| Lines 162-163: property_value=None → computes it first. | `isinstance(rent, float); rent > 0` | Pass | 0.063s |
| Calculate monthly rent positive | `isinstance(rent, float); 800 <= rent <= 15_000` | Pass | 0.000s |
| Calculate property area all types | `area > 0` | Pass | 0.000s |
| Lines 128-129: invalid date format → random years_ago. | `isinstance(price, float)` | Pass | 0.000s |
| Lines 120-121: property_value=None → computes it first. | `isinstance(price, float); price > 0` | Pass | 0.063s |
| Calculate sale price positive | `isinstance(price, float); 50_000 <= price <= 8_000_000` | Pass | 0.000s |

| Test Description | Acceptance Criteria | Result | Time |
|---|---|---|---|
| Lines 135-137: sale_date 1-2 years ago. | `isinstance(price, float)` | Pass | 0.000s |
| Lines 139-140: sale_date ¿ 2 years ago. | `isinstance(price, float)` | Pass | 0.000s |
| Lines 124-128: sale_date within 1 year. | `isinstance(price, float)` | Pass | 0.000s |
| Line 60-61: datetime → isoformat string. | `"2024-01-01" in result` | Pass | 0.000s |
| Lines 64-65: np.floating → float. | `"3.14" in result` | Pass | 0.000s |
| Lines 62-63: np.integer → int. | `result == "42"` | Pass | 0.000s |
| Lines 66-67: np.ndarray → list. | `result == [1, 2, 3]` | Pass | 0.000s |
| Line 68: super().default() raises TypeError for unknown type. | — | Pass | 0.000s |
| Lines 510-517: each fallback location has lat/lon/name/elevation. | `"lat" in loc`; `"lon" in loc` (+2 more) | Pass | 0.000s |
| Lines 499-518: fallback generates right number of locations. | `len(locs) == 5` | Pass | 0.000s |
| Lines 505-506: uses params.get_elevation if available. | `all(loc["elevation"] == 10.0 for loc in locs)` | Pass | 0.000s |
| Lines 608-612: generate_properties returns result dict. | `isinstance(result, dict)`; `"data" in result` (+1 more) | Pass | 0.615s |
| Lines 610-612: generate_properties creates output file. | `result["file_path"].exists()` | Pass | 0.380s |
| Calculate mortgage financials returns dict | `isinstance(result, dict)`; `"loan_amount" in result` (+2 more) | Pass | 0.000s |
| Generate financial data keys | `"mortgage_id" in data`; `"loan_amount" in data` (+1 more) | Pass | 0.000s |
| Interest rate range | `0.01 <= data["interest_rate"] <= 0.15` | Pass | 0.000s |
| Loan amount less than value | `data["loan_amount"] <= info["property_value"] * 1.05` | Pass | 0.000s |
| Mortgage id format | `re.match(r"^MORT-[a-f0-9]{8}$", data["mortgage_id"])` | Pass | 0.000s |
| Quality consistency check | `isinstance(corrected, dict)` | Pass | 0.000s |
| Calculate insurance premium positive | `isinstance(premium, (int, float))`; `premium > 0` | Pass | 0.066s |
| Calculate property area positive | `isinstance(area, (int, float))`; `area > 0` | Pass | 0.000s |
| Calculate property value in range | `50_000 <= value <= 10_000_000` | Pass | 0.639s |
| Generate bedrooms detached | `isinstance(beds, int)`; `1 <= beds <= 7` | Pass | 0.000s |
| Generate bedrooms flat | `isinstance(beds, int)`; `1 <= beds <= 4` | Pass | 0.000s |

| Test Description | Acceptance Criteria | Result | Time |
|---|---|---|---|
| Generate postcode format | `isinstance(postcode, str); len(postcode) >= 5` | Pass | 0.000s |
| Generate field value decimal | `isinstance(val, (int, float))` | Pass | 0.131s |
| Generate field value text | `isinstance(val, str)` | Pass | 0.064s |
| Coordinates in thames range | `51.0 <= loc["lat"] <= 52.0; -1.0 <= loc["lon"] <= 1.0` | Pass | 1.895s |
| Processing stats | `stats["successful_properties"] == 5; stats["failed_properties"] == 0` | Pass | 0.958s |
| Output file exists | `result["file_path"].exists()` | Pass | 0.935s |
| Output file is valid json | `isinstance(data, dict)` | Pass | 0.941s |
| Generate correct count | `len(result["data"]["properties"]) == 5` | Pass | 0.936s |
| Generate returns expected keys | `"data" in result; "file_path" in result` (+1 more) | Pass | 0.925s |
| Property id format | `re.match(r"^PROP-[a-f0-9]{8}$", pid)` | Pass | 1.016s |
| Property ids unique | `len(ids) == len(set(ids))` | Pass | 1.832s |
| Construction year in range | `1600 <= year <= 2026` | Pass | 0.000s |
| Property id format | `re.match(r"^PROP-[a-f0-9]{8}$", meta["property_id"])` | Pass | 0.062s |
| Returns dict with required keys | `"property_id" in meta; "property_type" in meta` | Pass | 0.066s |
| Lines 73-75: generate_grid_reference returns a non-empty string. | `isinstance(ref, str); len(ref) > 4` | Pass | 0.000s |
| Lines 50-52: generate_owner_name returns 'Firstname Lastname'. | `len(parts) == 2; all(p[0].isupper() for p in parts)` | Pass | 0.000s |
| Lines 43-45: generate_past_date returns YYYY-MM-DD string. | `isinstance(result, str); len(parts) == 3` | Pass | 0.000s |
| Custom days_range is respected (result is a past date). | `dt < datetime.now()` | Pass | 0.000s |

## 1.1 Test Document History

| Date | Version | Description | Author |
|---|---|---|---|
| 05-Mar-2026 | 1.0 | Initial test suite (28 tests) | D. Kelly |
| 08-Mar-2026 | 1.1 | 25 test(s) added; 28 test(s) removed (total: 25) | D. Kelly |
| 08-Mar-2026 | 1.2 | 4 test(s) added (total: 29) | D. Kelly |
| 09-Mar-2026 | 1.3 | 12 test(s) added (total: 41) | D. Kelly |
| 21-Mar-2026 | 1.4 | 14 test(s) added (total: 55) | D. Kelly |